MACHINE LEARNING ASSIGNMENTS

Linear Regression · Logistic Regression

Course	Machine Learning Fundamentals
Level	Undergraduate / Postgraduate
Total Assignments	2 (One per Algorithm)
Submission Format	Jupyter Notebook (.ipynb) + PDF Report

Note: Read each section carefully before starting. Follow all tasks in order.

ASSIGNMENT 1 — LINEAR REGRESSION

1. What is Linear Regression?

Linear Regression is one of the oldest and most widely used supervised machine learning algorithms. It models the relationship between one or more **independent variables** (features / predictors) and a **continuous dependent variable** (target / output) by fitting a straight line (or hyperplane in higher dimensions) through the data points.

Simple Linear Regression equation:

$$y = b_0 + b_1 \cdot x + \varepsilon$$

Multiple Linear Regression equation:

$$y = b_0 + b_1 x_1 + b_2 x_2 + \ldots + b_n x_n + \varepsilon$$

y = Predicted output (dependent variable)

b_0 = Bias / Intercept term

b_1 … b_n = Coefficients (weights) learned during training

x_1 … x_n = Input features (independent variables)

ε = Error term (residual — the difference between actual and predicted value)

2. Why Linear Regression?

Despite being a simple algorithm, Linear Regression remains extremely important in both academia and industry for several compelling reasons:

Interpretability	Coefficients directly tell you how much each feature contributes to the prediction, making the model easy to explain to non-technical stakeholders.
Computational Efficiency	Training is fast even on large datasets. The closed-form solution (Normal Equation) or Gradient Descent converges quickly.
Baseline Model	Linear Regression serves as the go-to baseline. If a complex model cannot beat it, something is wrong with the data or the approach.
Statistical Foundation	Well-studied statistical properties allow hypothesis testing, confidence intervals, and p-value analysis for feature significance.
Works Well on Linear Data	When the true relationship between variables is approximately linear, it achieves performance comparable to far more complex models.
Feature Engineering Insight	Residual analysis reveals patterns, outliers, and heteroscedasticity, guiding further feature engineering steps.

3. Key Assumptions of Linear Regression

Linear Regression makes the following assumptions about the data:

Linearity: The relationship between X and y is linear.

Independence: Observations are independent of each other.

Homoscedasticity: The residuals have constant variance across all levels of the independent variable.

Normality: Residuals are approximately normally distributed.

No Multicollinearity: Independent features are not highly correlated with each other.

4. Evaluation Metrics

Metric	Formula	Interpretation
MAE (Mean Absolute Error)	Mean(|y - ■|)	Average magnitude of errors — robust to outliers
MSE (Mean Squared Error)	Mean((y - ■)^2)	Penalises large errors more heavily
RMSE	sqrt(MSE)	Same unit as y — easier to interpret

R^2 Score	$1 - SS_{res}/SS_{tot}$	1 = perfect fit; 0 = baseline mean model
Adjusted R^2	Penalises extra features	Use when comparing models with different feature counts

5. Recommended Kaggle Datasets for Linear Regression

Download any **one** of the following datasets from Kaggle. Search for the dataset name on **kaggle.com/datasets**.

#	Dataset Name	Kaggle Link / Search Term	Target Variable
1	**House Prices — Ames Housing**	*Search: 'House Prices Advanced Regression Techniques'*	SalePrice (House price in USD)
2	**Boston Housing Dataset**	*Search: 'Boston Housing Dataset'*	MEDV (Median house value)
3	**Car Price Prediction**	*Search: 'Car Price Prediction Multiple Linear Regression'*	Price (Car selling price)
4	**Student Performance**	*Search: 'Student Performance Data Set'*	G3 (Final exam score)
5	**Insurance Premium Prediction**	*Search: 'Medical Cost Personal Datasets'*	charges (Insurance cost in USD)

6. Assignment Tasks

Part A — Data Exploration & Preprocessing

Task 1: Load the dataset and display the first 10 rows. Report the shape, column names, and data types.

Task 2: Perform Exploratory Data Analysis (EDA): plot histograms, box plots, and a correlation heatmap.

Task 3: Identify and handle missing values. Justify the imputation strategy chosen.

Task 4: Detect and treat outliers using the IQR method or Z-score. Show before/after distributions.

Task 5: Encode any categorical variables. Apply feature scaling (StandardScaler or MinMaxScaler) where necessary.

Part B — Model Building

Task 1: Split the data into Training (80%) and Testing (20%) sets using a fixed random_state.

Task 2: Train a Simple Linear Regression model using one feature that shows the highest correlation with the target.

Task 3: Train a Multiple Linear Regression model using all relevant features.

Task 4: Implement Gradient Descent from scratch (without sklearn) and compare results with the sklearn model.

Task 5: Visualise the regression line for Simple Linear Regression and the predicted vs actual plot for Multiple.

Part C — Evaluation & Analysis

Task 1: Compute MAE, MSE, RMSE, and R^2 Score for both train and test sets.

Task 2: Plot the residuals vs fitted values. Check for patterns that suggest model violations.

Task 3: Apply polynomial features (degree=2) and compare the R^2 score with the linear model.

Task 4: Use Lasso and Ridge Regression and discuss the effect of regularisation on model coefficients.

Task 5: Write a conclusion (min 200 words) explaining your findings, limitations, and possible improvements.

7. Deliverables

✓ **Jupyter Notebook** (.ipynb) with all code, outputs, and markdown explanations

✓ **PDF Report** (exported from notebook) containing all plots and a written analysis

✓ **README.md** describing the dataset chosen and how to run your notebook

✓ All files zipped and named: **Assignment1_LinearRegression_YourName.zip**

ASSIGNMENT 2 — LOGISTIC REGRESSION

1. What is Logistic Regression?

Despite its name, Logistic Regression is a **classification** algorithm, not a regression algorithm. It estimates the probability that a given input belongs to a particular class by applying the **Sigmoid (logistic) function** to a linear combination of input features. The output is always between 0 and 1, which can be interpreted as a probability.

Logistic / Sigmoid Function:

$$\sigma(z) = 1 / (1 + e^{-z}) \text{ where } z = b_0 + b_1x_1 + \ldots + b_nx_n$$

Decision boundary:

Predict Class 1 if $\sigma(z) \geq 0.5$, Predict Class 0 if $\sigma(z) < 0.5$

σ**(z)** = Probability output in the range [0, 1]

z = Linear combination of input features (same as Linear Regression's output)

$\mathbf{b_0}$ = Intercept/bias; $\mathbf{b_1 \ldots b_n}$ = Learned weights

e = Euler's number (≈ 2.718)

The model outputs a **probability**, not a class directly — a threshold converts it to a class label.

2. Why Logistic Regression?

Logistic Regression is one of the most powerful yet simple classification algorithms:

Probabilistic Output	Returns calibrated probability scores, not just class labels, enabling risk-based decisions.
Highly Interpretable	Coefficients can be exponentiated to obtain Odds Ratios — a concept widely used in medicine and social science.
No Assumption of Normality	Unlike Linear Discriminant Analysis, Logistic Regression does not assume features are normally distributed.
Efficient Training	Uses Maximum Likelihood Estimation (MLE) optimised via gradient descent — very fast on large datasets.
Works with Multiple Classes	Can be extended to multi-class problems via One-vs-Rest (OvR) or Softmax (Multinomial Logistic Regression).

Strong Baseline Classifier	Often used as the benchmark before trying complex models like SVMs, Random Forests, or Neural Networks.

3. Key Concepts in Logistic Regression

Understanding these concepts is critical before implementing the model:

Log-Odds (Logit): The natural logarithm of the odds ratio — logit(p) = ln(p / 1-p). Logistic Regression models this as linear in the features.

Maximum Likelihood Estimation (MLE): The training process maximises the likelihood of observing the actual class labels given the input features.

Binary Cross-Entropy Loss: The loss function used — Loss = –[y·log(■) + (1–y)·log(1–■)]. Penalises confident wrong predictions heavily.

Regularisation (L1/L2): L1 (Lasso) produces sparse models; L2 (Ridge) shrinks all coefficients — both prevent overfitting.

Threshold Tuning: The default threshold of 0.5 can be adjusted based on the business cost of false positives vs false negatives.

Class Imbalance: When classes are unequal in size, use class_weight='balanced', SMOTE oversampling, or under-sampling.

4. Evaluation Metrics for Classification

Metric	Formula / Description	When to Use
Accuracy	Correct Predictions / Total Predictions	Balanced classes only
Precision	TP / (TP + FP)	When false positives are costly
Recall (Sensitivity)	TP / (TP + FN)	When false negatives are costly
F1-Score	2 × (Precision × Recall) / (Precision + Recall)	Imbalanced classes
AUC-ROC	Area under ROC curve [0–1]	Model ranking ability overall
Log Loss	Mean cross-entropy loss	Probabilistic evaluation

5. Recommended Kaggle Datasets for Logistic Regression

Download any **one** of the following datasets from Kaggle. Search for the dataset name on **kaggle.com/datasets**.

#	Dataset Name	Kaggle Link / Search Term	Target Variable
1	**Titanic — Survival Prediction**	*Search: 'Titanic Machine Learning from Disaster'*	Survived (0 = No, 1 = Yes)
2	**Breast Cancer Wisconsin**	*Search: 'Breast Cancer Wisconsin Diagnostic'*	diagnosis (M = Malignant, B = Benign)
3	**Heart Disease Prediction**	*Search: 'Heart Disease UCI'*	target (0 = No disease, 1 = Disease)
4	**Diabetes Prediction**	*Search: 'Pima Indians Diabetes Database'*	Outcome (0 = No diabetes, 1 = Diabetes)
5	**Customer Churn Prediction**	*Search: 'Telco Customer Churn'*	Churn (Yes/No)

6. Assignment Tasks

Part A — Data Exploration & Preprocessing

Task 1: Load the dataset and display the first 10 rows. Report the shape, column names, and data types.

Task 2: Analyse class distribution — plot a bar chart of target classes and report imbalance ratio.

Task 3: Handle missing values with appropriate strategies. Encode categorical variables (Label/One-Hot Encoding).

Task 4: Perform feature scaling using StandardScaler. Justify why scaling matters for Logistic Regression.

Task 5: Visualise pairplots and correlation heatmap, colour-coding by class label.

Part B — Model Building

Task 1: Split data into 80% training and 20% testing sets with stratified sampling (stratify=y).

Task 2: Train a Logistic Regression model with default hyperparameters using sklearn.

Task 3: Implement Logistic Regression with Gradient Descent from scratch without using sklearn's LogisticRegression class.

Task 4: Train models with L1 (Lasso) and L2 (Ridge) regularisation. Compare the resulting coefficients.

Task 5: If class imbalance is present, apply SMOTE or class_weight='balanced' and retrain the model.

Part C — Evaluation & Analysis

Task 1: Generate a Confusion Matrix (with a heatmap visualisation) for both train and test predictions.

Task 2: Compute Accuracy, Precision, Recall, F1-Score, and Log Loss. Present in a comparison table.

Task 3: Plot the ROC Curve and report the AUC score. Explain what a higher AUC means.

Task 4: Tune the decision threshold (try 0.3, 0.5, 0.7) and analyse the trade-off between Precision and Recall.

Task 5: Write a conclusion (min 200 words) comparing the regularised vs unregularised models and any class imbalance handling.

7. Deliverables

✓ **Jupyter Notebook** (.ipynb) with all code, outputs, and markdown explanations

✓ **PDF Report** (exported from notebook) containing all plots and a written analysis

✓ **README.md** describing the dataset chosen and how to run your notebook

✓ All files zipped and named: **Assignment2_LogisticRegression_YourName.zip**

8. Grading Rubric (Applies to Both Assignments)

Component	Marks
Data Exploration & Preprocessing (EDA, handling nulls, encoding)	20
Model Implementation (both scratch & sklearn)	25
Evaluation Metrics (correct computation and interpretation)	20
Visualisations (quality, labels, and relevance)	15
Written Analysis / Conclusion	10
Code Quality (comments, structure, reproducibility)	10
TOTAL	**100**

Good luck! Focus on understanding the mathematics behind each algorithm, not just running the code. The goal is to build intuition that will help you tackle more complex ML problems in the future.